Advanced Data Analysis

Mike Nguyen

2021-01-18

Contents

Chapter 1

Prerequisites

```
install.packages("bookdown")
# or the development version
# devtools::install_github("rstudio/bookdown")
```

This book's skeleton is based on several courses. Hence, I'd like to credit professors accordingly.

Course	Professor
Analyzing Unstructured Data	Kunpeng Zhang
Deep Learning	Yann LeCun

Compare to the last book, this book should differ as follows:

Part I

MACHINE LEARNING

Chapter 2

Introduction

Resources

- INTRODUCTION TO MACHINE LEARNING

3 types of data

- Structured Data (tables form)
- Semi-structured Data (e.g., XML, JSON, HTML)
- Unstructured Data
  - Texts
  - Images: Videos, photos,
  - Networks: Social Networks

This section based on notes in "Analyzing Unstructured Data" course taught by professor Kunpeng Zhang.

- ML is concerned with the the development, the analysis, and the application of algorithm that allow computers to learn
- Learning:
  - A computer learns if it improves its performance at some tasks with experience (i.e., data)
  - Extracting a model of system from the sole observation (or the simulation)
  - A model = some relationships between the variables used to describe the system.

- Goals:
  - Make prediction
  - Understand the underlying mechanisms.
- Learning if useful when:
  - Human expertise does not exist (navigating on Mars)
  - Humans are unable to explain their expertise (speech recognition)
  - Solution changes in time (routing on a computer network)
    Solution needs to be adapted to particular cases (user biometrics)
- Applications in different domains:
  - Retail: Market basket analysis, customer relationship management (CRM)
  - Finance: Credit scoring, fraud detection.
  - Manufacturing: Optimization, troubleshooting
  - Medicine: medical diagnosis
  - Telecommunications: Quality of service optimization, routing
  - Bioinformatics: Motifs, alignment WEb mining, search engines.
- Steps
  - Data generation: data types, sample size
  - Preprocessing: normalization, missing values, feature selection/extraction
  - Machine Learning: Hypothesis, choice of learning paradigm/algorithm.
  - Hypothesis validation: cross-validation, model deployment.
- Types of machine learning:
  - Supervised: manually label variables
  - Unsupervised:
- Training and testing
  - Training is the process of making the machine learn

  - No free lunch:
    * Training set and testing set come from the same distribution
    * Need to make some assumptions.

- Factors affecting the performance:
  - Types of training provided
  - The form and extend of prior knowledge
  - Type of feedback provided
  - The learning algorithms used

- Two factors:
  - Modeling
  - optimization

- Algorithms:
  - ML depends on algorithms.
  - The algorithms control the search to find and build the knowledge structures
  - The learning algorithms should extract useful information from training examples.

- Types of learning (algorithms):
  - Supervised learning ($\{X_n \in R^d, y_n \in R\}_{n=1}^N$)
    * Prediction
    * Classification (discrete labels), regression (real values)
  - Unsupervised learning ($\{X_n \in R^d\}_{n=1}^N$)
    * Clustering
    * Probability distribution estimation
    * Finding association (in features)
    * Dimension reduction
  - Semi-supervised learning
  - Reinforcement learning

- Decision making (robot, chess machine)

Chapter 3

Supervised Learning

From the data, find a function f of the inputs that best approximates the outputs.

$$\hat{Y} = f(X_1, .., X_n)$$

- Predictive: Make predictions for a new object described by its attributes
- Informative: help understand the relationship between inputs and outputs

Divide data into

- Training set: train model
- validation set: tune hyper parameters
- Test set: evaluation of the model for generalization error.

Learning algorithm

- defined by
  - a family of candidate models (= **hypothesis space H**)
  - a quality measure for a model
  - an optimization strategy
- takes as input a leaning sample and outputs a function h in H of maximum quality

Model selection

- Why not choose the model that minimizes the error rate on the learning sample? (**re-substitution error**)

- How well are you going to predict future data drawn from the same distribution? (**generalization error**)
- Evaluation on test set
  1. Randomly choose a percentage (e.g., 30%) of the data to be in a test set

  2. The remainder is training set

  3. Learn the model from the training set

  4. Estimate its future performance on the test set.

- Problems:
  - overfits: occurs when the learning algorithm starts fitting noise.

  - unfits

- Evaluation on test set:
  - Good:
    * Very simple

    * Computationally efficient

  - Bad:
    * Waste data

    * very unstable when database is small

- Another evaluation: **Leave-one-out cross validation**
  - For k = 1 to N
    * remove the k-th object from the learning sample

    * learn the model on the remaining objects

    * apply the model to get a prediction for the k-th object

  - report the proportion of mis-classified objects

- Good:
  - Do not waste the data (you get an estimate of the best method to apply to N-1 data)

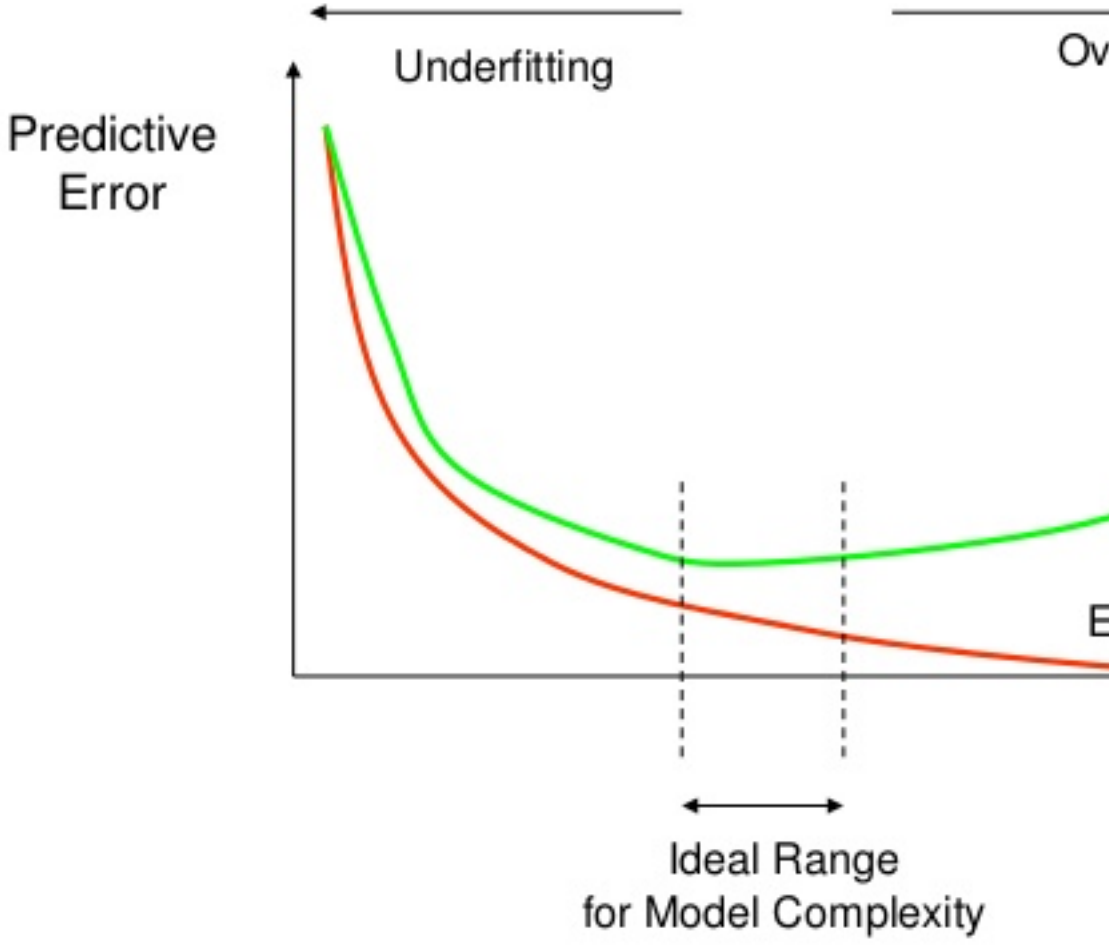
- Bad:

  - Computationally intensive (train N models)
  - high variance.

- Another evaluation: **K-fold cross validation**:
  - Break up data into k folds
  - For each fold
    * Choose the fold as a temporary test set
    * Train on k-1 folds, compute performance on the test fold.
  - Report average performance of the 10 runs.
  - randomly partition the dataset into k subsets (e.g., 10)
  - For each subset:
    * learn the model on the objects that are not in the subset
    * compute the error rate on the points in the subset. Report the mean error rate over the k subsets.
  - When k = the number of objects -> leave-one-out-cross validation.

Rule-of-thumb for evaluation

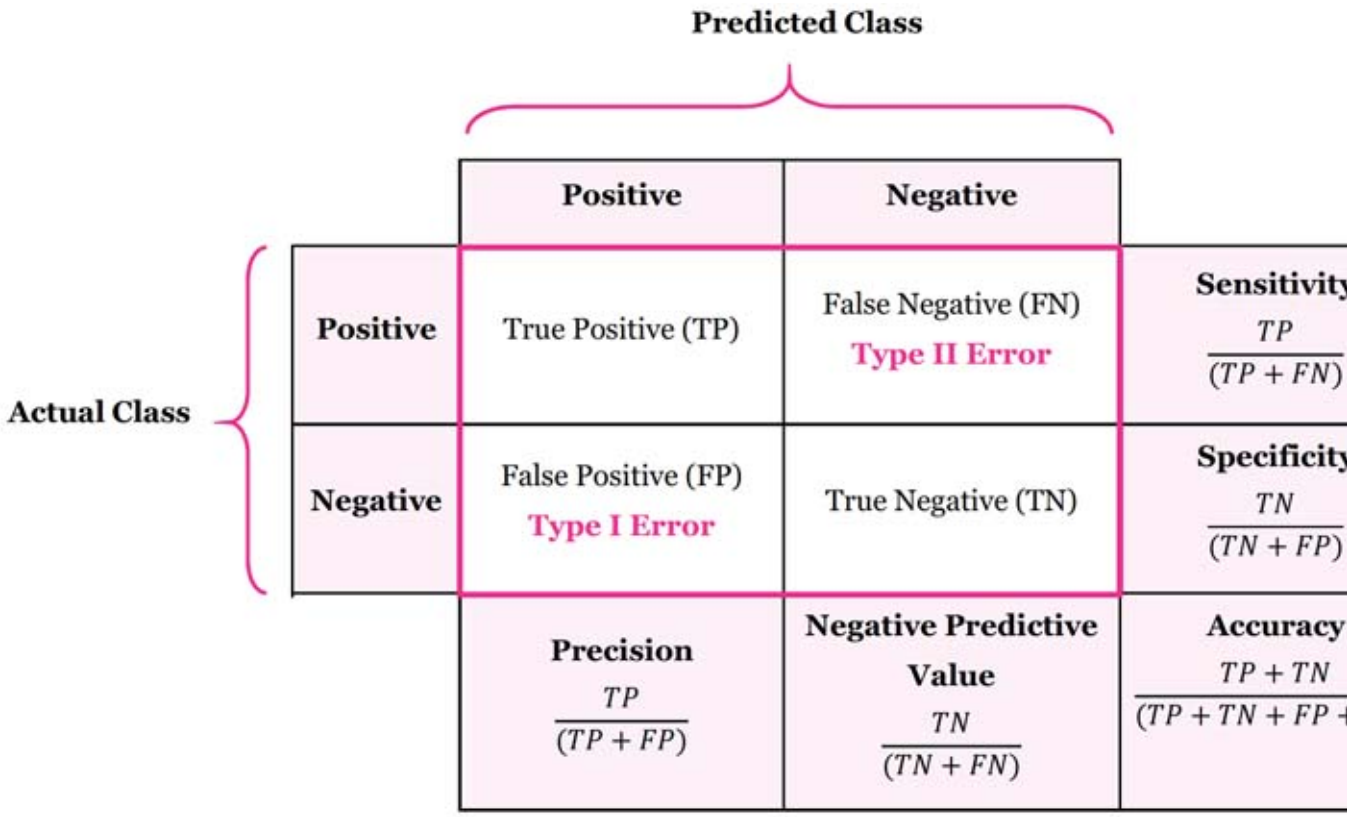
- Lots of data (>1000) test set validation
- small data (100-1000): 10-fold CV
- very small data (<100) leave one out CV.


How Overfitting affects Prediction
Underfitting
Predictive
Error
Ideal Range
for Model Complexity

Use the testing set to estimate the performance of the final model

Performance measures

- The error rate is no the only way to assess a predictive model.
- In binary classification, results can be summarized in a contingency table (i.e., confusion matrix)



Various criteria

$$\text{Error rate} = \frac{FP+FN}{N+P} \quad \text{Accuracy} = \frac{TP+TN}{N+P} = 1 - \text{Error rate} \quad \text{Sensitivity or recall} = \frac{TP}{P} \quad \text{Specificity} = \frac{TN}{TN+FP} \quad \text{Pr}$$

ROC and precision/recall curves

- Each point corresponds to a particular choice of the decision threshold.

Also known as robustness check

You want large area under the curve

Compassion of learning models

- Three main criteria:
  - Accuracy: Measures by the generalization error (estimated by CV)
  - Efficiency: Computing times and scalabiility for learning and testing
  - Interpretability: Comprehension brought by the model about the input-output relationship.
- Depends on the application, you have to make trade-off.

Learning Techniques

- Supervised learning categories and techniques
  - Linear classifier (numerical functions):
    - Perceptron
    - Logistic regression
    - Support vector machine (SVM): linear to nonlinear: feature transofrm and kernel function
    - Ada-line
    - Multi-layer perceptron (MLP)
  - Parametric (Probabilistic functions)
    - Naive Bayes, Gaussian discriminant analysis (GDA), Hidden Markov models (HMM), probabilistic graphical models
  - Non-parametric (Instance-based functions)
    - K-nearest neighbors, Kernel regression, Kernel density estimation, local regression.
  - Non-metric (symbolic functions)
    - Classification and regression tree (CART), decision tree
  - Aggregation
    - Bagging (bootstrap + aggregation), Adaboost, Random forest.
- Unsupervised learning categories and techniques:
  - Clustering
    - K-means clustering
    - Spectral clustering
  - Density Estimation

        * Gaussian mixture model (GMM)
        * Graphical models
    - Dimensionality reduction
        * Principal component analysis (PCA)
        * Factor analysis

- Semi-supervised learning
    - goal: exploit both labeled and unlabeled data to build better models than using each one alone
    - Self-training:
        * Iteratively label some unlabeled examples with a model learned from the previously labeled examples
        * Active learning.

- Reinforcement learning
    - Learning from interactions
    - Goal: learn to choose sequence of actions (=policy) that maximizes $r_0 + \gamma r_1 + \gamma^2 r_2 + ...$ where $0 \leq \gamma < 1$

3.1 Decision Tree

- Is a tree-structured plan of a set of attributes to test to predict the output.
- Give one attribute, try to predict the value of new people's attribute by means of some other available attribute
- Input attributes:
    - **d** features/ attributes $x^{(1)}, x^{(2)}, ..., x^{(d)}$
    - Each $x^{(j)}$ has domain O_j
        * Categorical $O_j = (read, blue)$
        * Numerical: $H_j = (0, 10)$
    - Y is output variable with domain O_Y:
        * Categorical: Classification
        * Numerical: regression

- Data D:
  - n examples (x_i, y_i) where
    - x_i is a d-dim feature vector
    - $y_i \in O_Y$
- Decision trees:
  - Split the data at each internal node
  - Each leaf node makes a prediction

How to construct a tree

- Find best split
- Stopping Criteria
- Find Prediction

3.1.1 Find best split

Pick an attribute and value that optimizes some criterion
+ Regression: Purity + Classification: Information Gain: Measures how much a given attribute X tells us about the class Y.

Information Gain? Entropy

The entropy of X: $H(X) = -\sum_{j=1}^{m} p_j log p_j$

- "High entropy": X is from a uniform distribution
  - A histogram of the frequency distribution of values of X is flat
- "Low Entropy": X is from varied distribution
  - A histogram of the frequency distribution of values of X would have many lows and one or two highs.
- Def: Information Gain:
  - $IG(Y|X) = H(Y) - H(Y|X)$ I must transmit Y. How many bits on average would it save me if both ends of the line knew X?

3.1.2 Stopping Criteria

- When the leaf is "pure": the target variable does not vary too much: Var(Y) < threshold.
- When number of examples in the leaf is too small.
- Could also use information as as a stopping criteria.

3.1.3 Find Prediction

- Regression: Predict average y of the examples
- Classification: Predict most common y of the examples the leaf.

3.2 Random Forest

Ensemble methods

- A single decision tree does not perform well.
- But, it is super fast.
- What if we learn multiple trees?

Bagging

- If we split the data in random different ways, decision trees give different results, high variance.
- Bagging: Bootstrap aggregating is a method that result in low variance.
- If we had multiple realizations of the data (or multiple samples) we could calculate the prediction multiple times and take the average of the fact that averaging multiple onerous estimations produce less uncertain results
- say for each sample b, we calculate $f^b(x)$, then $\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$
- Bootstrap: Construct B (hundreds) of trees. Learn a classifier for each bootstrap sample and average them. Very effective.
- Reduces overfitting (variance)
- Could use one classifier, or multiple classifiers.
- easy to parallelize
- Variable Importance Measures
  - Bagging results in improved accuracy over prediction using a single tree
  - Unfortunately, difficult to interpret the resulting model. Bagging improves prediction accuracy at the expense of interpretability.
- Issues:
  - Each tree is identically distributed (i.d)
    - The expectation of the average of B such trees

    - The bias of bagged trees is the same as that of individual trees
  - An average of B i.i.d random variables, each with variance σ^2, has variance σ^2/B. If i.d. (identical but not independent) and pair correlation ρ is present, then the variance is : $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. As B increases, the second term disappears but the first term remains.

- Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors. Then all bagged trees will select the strong predictor at the top tree and therefore all trees will look similar.
  - We can penalize the splitting with a penalty term that depends on the number of times a predictor is selected at a given length.
  - restrict how many items a predictor can be used.
  - allow a certain number of predictors.

- Since we want i.i.d so that the bias to be the same and variance to be less.
  - in bagging, we build a number of decision trees on bootstrapped training samples each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors (if m = p, then this is bagging).

Random Forest software

Random Forests Algorithm

For b = 1 to B:

a. draw a bootstrap sample Z * of size N from the training data.

b. Grow a random-forest tree to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
   - Select m variables at random from the p variables.
   - Pick the best variable/split-point among the m.
   - Split the node into two daughter nodes.

Output the ensemble trees.

	Regression	Classification
To make a prediction at a new point x	average the results	majority vote

	Regression	Classification
Tuning	The default value for m is $p/3$ and the minimum node size is 5	The default value for m is $\sqrt{p}$ and the minimum node size is one

Another issue:

When the number of variables is large, but the fraction of relevant variables is small, random forests are likely to perform poorly when m is small. Because: At each split the chance can be small that the relevant variables will be selected.

Example: with 3 relevant and 100 not so relevant variables the probability of any of the relevant variables being selected at any split is approx 0.25.

random forests "cannot overfit" the data because the number of trees (B) does not mean increase in the flexibility of the model.

Always include Random Forest (ensembles) and XGBoost (like bagging)

Alternative

- Support vector machine (SVM) dont usually use this because of computational reasons and because it is only 1 model.

Chapter 4

Unsupervised Learning

Unsupervised learning tries to find regularities in the data without guidance about inputs and outputs

Unsupervised learning categories and techniques:

- Clustering
  - K-means Clustering
  - Spectral Clustering
- Density Estimation
  - Gaussian mixture model (GMM)
  - Graphical models
- Dimensionality reduction
  - Principal component analysis (PCA)
  - factor Analysis

4.1 Clustering

Clustering is used to find similar groups in data (i.e., clusters)

No class value denoting an a priori grouping (hence, Unsupervised Learning)

Clustering algorithm

- Partitional clustering
- Hierarchical clustering

A distance (similarity, or dissimilarity) function

Clustering quality

- Inter-clusters distance -> maximized
- Intra-clusters distance -> minimized

which can be use to judge good clustering

- Internal criterion: A good clustering will produce high quality clusters in which:
  - The intra-class (intra-cluster) similarity is high
  - The inter-class similarity is low.
  - The measured quality of a clustering depends on both the document representation and the similarity measure used.
- External criteria:
  - Quality measured by its ability to discover some or all of the hidden patterns or latent classes in gold standard data.
  - Assesses a clustering with respect to **ground truth**
  - Assume documents with C gold standard classes, while our clustering algorithm produce cluster $w_1, w_2, ..., w_k$ with $n_1, ..., n_k$ members.
  - Simple measure: **purity**, the ratio between the dominant class in the cluster π_i and the size of cluster w_i

$$Purity(w_i) = \frac{1}{n_i} max_j(n_{ij}) j \in C$$

  - others are entropy of classes in clusters (or mutual inforamtion between classes and clusters).

The quality of a clustering result depends on

- The algorithm
- the distance function
- the application

4.1.1 Similarity Measures

It's your choice of how to define similarity

(Dis)similarity measure

- Equivalently, we can use dissimilarity measures
- Jagota defines a dissimilarity measure as a function f(x,y) such that f(x,y) > f(w,z) if and only if x is less similar to y than w is to z.
  Also known as **pair-wise** measure

Distance functions

Different distance functions for

- Different types of data
  - Numeric data
  - Nominal data
- Different applications

Numeric data

- $dist(x_i, x_j)$,where
  - $dist(x_j, x_j) = ((x_{i1} - x_{j1})^h + (x_{i2} - x_{j2})^h + ... + (x_{ir} - x_{jr})^h)^{\frac{1}{h}}$
  - x_i and x_j are data points (vectors)
- Euclidean distance
  - If h = 2, it is the Euclidean distance: $dist(x_j, x_j) = ((x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + ... + (x_{ir} - x_{jr})^2)^{\frac{1}{2}}$
  - Weighted Euclidean distance: $dist(x_j, x_j) = (w_1(x_{i1} - x_{j1})^2 + w_2(x_{i2} - x_{j2})^2 + ... + w_r(x_{ir} - x_{jr})^2)^{\frac{1}{2}}$
  - Squared Eulidean distance: to place progressively greater weight on data points that are further apart: $dist(x_j, x_j) = (x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + ... + (x_{ir} - x_{jr})^2$
- Manhattan distance
  - If h = 1, it is the Manhattan distance: $dist(x_j, x_j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + ... + |x_{ir} - x_{jr}|$
- Minkowski distance (h is positive integer)
  - $dist(x_j, x_j) = ((x_{i1} - x_{j1})^h + (x_{i2} - x_{j2})^h + ... + (x_{ir} - x_{jr})^h)^{\frac{1}{h}}$
  - 1st dimension, 2nd dimension, etc.
- Chebychev distance: define 2 data points as "different" if they are different on any one of the attributes
  - $dist(x_j, x_j) = max(|x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + ... + |x_{ir} - x_{jr}|)$

Binary and nominal Attributes

- Binary attribute: has 2 values or states but no ordering relationship (e.g., gender)

- We use a confusion matrix to introduce the distance function/measures
- Let the i-th and j-th data points be X_i and x_j (vectors)

Confusion matrix

		Data point j		
		1	0	
Data point i	1	a	b	a + b
	0	c	d	c + d
		a + c	b + d	a + b + c + d

- a: the number of attributes with the value of 1 for both data points
- b: The number of attributes with which $x_{if} = 1$ and $x_{jf} = 0$ where x_{if} is the value of f attribute of the data point x_i
- c: the number of attributes of which $x_{if} = 0$ and $x_{jf} = 1$
- d: the number of attributes with the value of 0 for both data points.

Symmetric binary attributes

- A binary attribute is symmetric if both of its states (0 and 1) have equal importance, and carry the same weights, e.g., Gender.
- distance function: **Simple Matching Coefficient**, proportion of mismatches of their values

$$dist(x_i, x_j) = \frac{b + c}{a + b + c + d}$$

Asymmetric binary attributes

- if one of the states is more important or more valuable than the other
  - By convention, state 1 represents the more important state.
  - Jaccard coefficient is popular measure

$$dist(x_i, x_j) = \frac{b + c}{a + b + c}$$

  - We can have some variations, adding weights

Nominal attributes: with more than 2 states or values.
* the common used distance measure is also based on the simple matching method.
* Given two data points x_i, x_j let the number of attributes be r, and the number of values that match in x_i and x_j be q. $dist(x_i, x_j) = \frac{r-q}{r}$

Data standardization

- In the Euclidean space, standardization of attributes is recommended so that all attributes can have equal impact on the computation of distances.
- standardize attributes: to force the attributes to have a common value range.
- Real data have different types:
  - Interval-scaled
  - symmetric binary
  - asymmetric binary
  - ratio-scaled
  - ordinal
  - nominal
- Convert to a single type:
  - To deal with mixed attributes:
    - Decide the dominant attribute type
    - Convert the other types to this type

4.1.2 K-mean Algorithm

K-means is a partitional clustering algorithm.

Let the set of data points (or instances) D to be $(x_1, ..., x_n)$, where $x_i = (c_{i1}, ..., x_{ir})$ is a vector in a real-valued space $X \subseteq R^r$ and r is the number of attributes (dimensions) in the data

The k-means algorithm partitions the given data into k clusters

- Each cluster has a cluster center (centroid)
- k is specified by user.

Steps

1. Randomly choose k data points (seeds) to be the initial centroids (cluster centers)
2. Assign each data point to the closest centroid
3. Re-compute the centroids using the current cluster memberships

4. If a convergence criterion is not met, go back to 2.

Convergence Criterion

1. No(or minimum) re-assignments of data points to different clusters,
2. No (or minimum) change of centroids, or
3. minimum decreases in the sum of squared error (SSE)

$$SSE = \sum_{j=1}^{k} \sum_{x \in C_j} dist(x.m_j)^2$$

- C is the j-th cluster,m is the centroid of cluster C_j (the mean vector of all the data points in C). and dist(x,m) is the distance between data point x and centroid m_j.

Pros

- Simple: easy to understand and implement
- Efficient: time complexity: O(tkn), where n is the number of data points, k is the number of clusters, and t is the number of iterations.
- Since both ka nd t are small, k-means is considered a linear algorithm
- Note: it terminates at a local optimum if SSE is used. The global optimum is hard to find due to complexity.

Cons

- The algorithm is only applicable if the mean is defined.
  - For categorical data, k-mode
- The user needs to specify k
- The algorithm is sensitive to outliers.
  - outliers could be errors in the data recording or some special data points with very different values.
- is not suitable for discovering clusters that are not hyper-ellipsoids (or hyper-spheres)

Solution

- To deal with outliers:
  - remove some data points int the clustering process that are much further away from the centroids than other data points.

  - Or perform random sampling. Since in sampling we only choose a small subset of the data points, the chance of selecting an outlier is very small.

- To deal with sensitive algorithm
  - Use different seeds

Chapter 5

Semi-superivsed learning

- Goal: exploit both labeled and unlabeled data to build better models than using each one alone

- Self-training
  - Iteratively label some unlabeled examples with a model learned from the previously labeled examples

  - Active learning

Chapter 6

Reinforcement learning

- Learning from interactions
- Goal: learn to choose sequence of actions (= policy) that maximizes $r_0 + \gamma r_1 + \gamma^2 r_2 + ...$

Chapter 7

Interpretable Machine Learning

https://christophm.github.io/interpretable-ml-book/ https://en.wikipedia.org/wiki/Explainable_artificial_intelligence https://arxiv.org/pdf/1905.08883.pdf https://arxiv.org/abs/2004.14545 https://www.kaggle.com/learn/machine-learning-explainability https://towardsdatascience.com/interperable-vs-explainable-machine-learning-1fa525e12f48

7.1 SHAP

https://github.com/slundberg/shap https://meichenlu.com/2018-11-10-SHAP-explainable-machine-learning/ https://bjlkeng.github.io/posts/model-explanability-with-shapley-additive-explanations-shap/

7.2 Video Classification

https://hackernoon.com/continuous-video-classification-with-tensorflow-inception-and-recurrent-nets-250ba9ff6b85 https://cloud.google.com/video-intelligence https://arxiv.org/pdf/1609.06782.pdf https://www.tubefilter.com/2019/02/12/taxonomy-youtube-videos-develop-original-content-that-works/ https://medium.com/swlh/converting-news-into-video-stories-5d8d4fc5a32b https://towardsdatascience.com/introduction-to-video-classification-6c6acbc57356 https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0228579

7.3 Image Memorability

(AMNET)

https://www.researchgate.net/publication/335603628_Image_Memorability_Using_Diverse_Visual_Features_and_Soft_Attention https://www.researchgate.net/publication/308812354_Deep_Learning_for_Image_Memorability_Prediction_the_Emotional_Bias https://arxiv.org/pdf/2005.02969.pdf https://github.com/dazcona/memorability https://arxiv.org/abs/1804.03115 https://www.researchgate.net/publication/324387176_AMNet_Memorability_Estimation_with_Attention http://ceur-ws.org/Vol-2283/MediaEval_18_paper_24.pdf https://jov.arvojournals.org/article.aspx?articleid=2771173 https://openaccess.thecvf.com/content_cvpr_2018/papers/Fajtl_AMNet_Memorability_Estimation_CVPR_2018_paper.pdf https://ieeexplore.ieee.org/document/8578764

https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7382237&tag=1 https://people.umass.edu/~yangzhou/

http://people.csail.mit.edu/khosla/projects/regionmem/ https://github.com/adikhosla/feature-extraction https://expertprogrammanagement.com/2019/12/elaboration-likelihood-model/

Part II

DEEP LEARNING

Chapter 8

Introduction

(LeCun et al., 2015) for a review. (LeCun et al., 2015) defines a deep-learning architecture as "a multilayer stack of simple modules, all (or most) of which are subject to learning, and many of which compute non-linear input-output mappings".

Chapter 9

Neural Network

Neuron

$$z = a_1 w_1 + .. + a_k w_k + b$$

Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Different connections lead to different network structures

The neurons have different values of weights and biases.
Fully connected Feedforward Network

Framework

Step 1: A set of function
Step 2: Goodness of function f
Step 3: Pick the best function

9.1 Step 1: A set of function

Deep = many hidden layers

Universality Theorem
Any continuous, function f

$$f : R^N \rightarrow R^M$$

can be realized by a network with one hidden layer (given enough hidden neurons).

Output layer

- Softmax layer as the output layer

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

  - where
    - σ = softmax
    - $\vec{z}$ = input vector
    - e^{z_i} = standard exponential function for input vector
    - K = number of classes in the multi-class classifier
    - e^{z_j} = standard exponential function for output vector
  - probability:
    - $1 > y_i > 0$
    - $\sum_i y_i = 1$
  - Monotonicity
  - Non-locality

9.2 Step 2: Goodness of function

Loss can be square error or cross entropy between the network output and target.

With softmax, the summation of all the outputs would be one.

A good function should make the loos of all examples as small as possible

Total Loss:

$$L = \sum_{r=1}^{R} l_r$$

Find a function in function set (the network parameters θ^*) that minimizes total loss.

9.3 Step 3: Pick the best function

Find network parameters θ^* that minimize total loss L.

Gradient Descent

Network parameters $\theta = (w_1, w_2, .., b_1, b_2, ...)$

- Pick an initial value for w
- Compute $\frac{\partial L}{\partial w}$,
  - if negative, increase w
  - if positive, decrease w
- $w_{new} = w_{old} - \eta \frac{partialL}{\partial w}$ where η is the learning rate.
- You have problems with:
  - plateau: $\frac{\partial L}{\partial w} \approx 0$
  - saddle point: $\frac{\partial L}{\partial w} = 0$
  - Stuck at local minima: $\frac{\partial L}{\partial w} = 0$
- solution:
  - Different initial point to reach different minima, so different results.

Backpropagation

- is an efficient way to compute $\partial L / \partial w$

9.4 Recipe of Deep learning

1. Not good result on training data

- Choosing proper loss
- Mini-batch
- New activation function
- Adaptive Learning Rate
- Momentum

2. Not good result on testing data

- Early Stopping
- Regularization
- Dropout
- Network Structure

9.4.1 Choosing proper loss

When using softmax output layer, choose cross entropy. Problem

9.4.2 Mini-batch

```
model.fit(x_train,y_train,batch_size = 100,nb_epoch = 20) # 100 examples in a mini-bat
# repeat 20 times
```

- Randomly initialize network parameters
- One epoch
  - Pick the 1st batch $L' = l^1 + l^{31} + ..$ Update parameters once.
  - Pick the 2nd batch $L'' = l^2 + l^{16} + ...$ Update parameters once
  - Until all mini-batches have been picked
- Repeat the above process
- Mini-batch is faster (with mini-batch. if there are 20 batches, update 20 times in one epoch).

9.4.3 New activation function

- Rectified Linear Unit (ReLU)

Reason:

1. Fast to compute
2. Biological reason
3. Infinite sigmoid with different biases
4. Vanishing gradient problem.

Parametric ReLU (PReLU) (Glorot et al., 2011)

ReLU - Variant

- Leaky ReLU
- Parametric ReLu

9.4.4 Adaptive Learning Rate

- Set the learning rate η carefully
- If learning rate is too large
- Total loss may not decrease after each update.
- If learning rate is too small
- Training would be too slow

- Popular and simple idea: reduce the learning rate by some factor every few epochs.
  - At the beginning, we are far from the destination, so we use larger learning rate
  - After several epochs, we are close to the destination, so we reduce the learning rate
  - E.g., 1/t decay $\eta^t = \eta\sqrt{t+1}$
- Learning rate cannot be one-size-fits-all
  - Giving different parameters different learning rates.
- η is unit-dependent (from 0 to 1)

Adagrad

- Original: $w \leftarrow w - \eta \partial L/\partial w$
- Adagrad: $w \leftarrow -\eta_w \partial L/\partial w$
  - where η_w is parameter dependent learning rate.

$$\eta_w = \frac{\eta}{\sqrt{\sum_{i=1}^{t}(g^i)^2}}$$

  - η is constant
  - g^i is $\partial L/\partial w$ obtained at the i-th update.
- Learning rate is smaller and smaller for all parameters
- Smaller derivatives, larger learning rate, and vice versa.

link

9.4.5 Momentum

Hard to find optimal network parameters

Movement = negative of $\partial L/\partial w$ + Momentum

```
# RMSProp (Advanced Adagrad) + Momentum
model.compile(loss='categorial_crossentropy',optimizer = SGD(lr=0.1),metrics = ['accuracy'])
model.compile(loss='categorial_crossentropy',optimizer=Adam(),metrics = ['accuracy'])
```

9.4.6 Early Stopping

- training data and testing data can be different.
- Learning target is defined by the training data

- The parameters achieving the learning target do not necessary have good results on the testing data.

9.4.7 Regularization

9.4.8 Dropout

Training

- each time before updating the parameters
  - each neuron has p% to dropout.
    - The structure of the network is changed.
  - Using the new network for training
    - For each min-batch, we resample the dropout neurons.

Testing

- No dropout
  - If the dropout rate training is p%, all the weights times 1-p%
  - Assume that the dropout rate is 50%. If a weight w = 1 by training, set w = 0.5 for testing.

Dropout is a kind of ensemble.

- training of dropout: M neurons $\rightarrow 2^M$ possible networks.
- Using one mini-batch to train one network
- Some parameters int eh network are shared.
- Training of dropout: assume dropout rate is 50%
- Testing of dropout: weight

```
model = Sequential()
model.add(Dense(input_dim = 28*28,output_dim = 500))
model.add(Activation('sigmoid'))
model.add(dropout(0.8))
model.add(Dense (output_dim=500))
model.add(Activation('sigmoid'))
model.add(dropout(0.8))
model.add(Dense(output_dim = 10))
model.add(Activation('softmax'))
```

9.4.9 Network Structure

Convolutional Neural Network (CNN)

- Widely used in image processing
- use in the first step for deep learning

The Whole CNN

- Property 1: Some patterns are much smaller than the whole image
- Property 2: The same patterns appear in different regions.
- Property 3: Subsampling the pixels will not change the object.

CNN in Keras: only modified the network structure and input format (vector to 3-D tensor)

```
model2.add(Convolution2D(25,3,3),input_shape= (1,28,28))
# 25 3x3 filters; 1:black/weight, 3: RFB, 28x28 pixels
model2.add(MaxPooling2D(2,2))
model2.add(Convolution2D(50,3,3))
model2.add(MaxPooling2D(2,2))
model2.add(Flatten())
model2.add(Dense(output_dim = 100))
model2.add(Activation('relu'))
model2.add(Dense(output_dim=10))
model2.add(Activation('softmax'))
```

will we always have to know our classes? Yes, and you have to retrain your model all the time. Or you could use
Transfer learning in deep learning

Neural topic modeling

graph representation learning

GNN

DGL

recurrent network

BERT model

Chapter 10

Overview

- What is deep learning? Input to output via layers of representation
- What are layers? A layer is a geometric transformation function on the data that goes through it Weights determine the data transformation behavior of a layer

Learning Representation

- Transforming input data into useful representation

The “deep” in deep learning

- multiple layers
- Other possibly more appropriate names for the field:
  - Layered representations learning
  - Hierarchical representations learning
  - Chained geometric transformation learning

New problem domains for R:

- Computer vision
- Computer speech recognition
- Reinforcement learning applications

How do we train deep learning models?

- Basic of machine learning algorithms
- Machine learning vs. statistical modeling
- MNIST example

  - Model definition in R
  - Layers of representation
- The training loop

Machine learning algorithms
Learning model parameters via exposure to many example data points

Statistics: Often focused on inferring the process by which data is generated
Machine Learning: Principally focused on predicting future data

Deep learning frontiers

- Computer vision
- Natural language processing
- Time series
- Biomedical

Chapter 11

Tensor Flow and R

TensorFlow APIs

- Keras API
- Estimator API
- Core API

11.1 R packages

TensorFlow APIs * keras: Interface for neural networks, with a focus on enabling fast experimentation. * tfestimators: Implementations of common model types such as regressors and classifiers.
* tensorflow: Low *level interface to the TensorFlow computational graph.* tfdatasets: Scalable input pipelines for TensorFlow models.

Supporting Tools * tfruns : Track , visualize, and manage TensorFlow training runs and experiments * tfdeploy : Tools designed to make exporting and serving TensorFlow models straightforward. * cloudml : R interface to Google Cloud Machine Learning Engine.

R interface to Keras

- Step by step example
- Keras layers
- Compiling models
- Losses, Optimizers, and Metrics
- More examples

11.2 Keras layers

65 layers available

- Dense layers: classic “fully connected” neural network layers
- Convolutional layers: ”Filters for learning local patterns in data
- Recurrent layers: Layers that maintain state based on on previously seen data
- Embedding layers: Vectorization of text that reflects semantic relationships between words

Chapter 12

Compiling models

Model compilation prepares the model for training by:
1. Converting the layers into a TensorFlow graph
2. Applying the specified loss function and optimizer
3. Arranging for the collection of metrics during training

Chapter 13

Losses, Optimizers, and Metrics

All available at Keras for R cheatsheet

Example of Image classificaiton

Chapter 14

NYU

Materials in this chapter is based on NYU Deep Learning

Deep in Deep Learning means multi-layers. "A deep network has several layers and uses them to build a hierarchy of features of increasing complexity"

Supervised Learning (= Function Optimization): With inputs and outputs, you train the machine to tweak its parameter to have the correct outputs from inputs. In "traditional machine learning", you have to engineer **feature extractor**, but in deep learning you can multi-layers feature extractor can be trained. And all layers are non-linear, because if they are linear, then all layers would be collapse into one layer.

Natural Data is compositional. Hence, it is efficiently representable hierarchically.

Revolutions in Deep Learning:

- Speech Recognition: 2010
- Image Recognition: 2013
- Natural Language Processing (NLP): 2015

Deep Learning = Learning Representations/ Features Representation means you turn raw data into something useful

Image Recognition:
Pixel, edge, texton, motif, part, object

Text
Character, group, word group, clause, sentence, story

Speech
Sample, spectral band, sound, phone, phoneme, word.

Difference between Deep Learning Support-Vector Machines (SVM)

SVM is a 2-layer neural net, where

- layer 1 = templates
- layer 2 = linear classifier.

where Deep Learning is "A deep network has several layers and uses them to build a hierarchy of features of increasing complexity"

Deep Machines are more efficient than SVM in representing certain classes of functions.

The Manifold Hypothesis "states that real-world high-dimensional data lie on low-dimensional manifolds embedded within the high-dimensional space"

Difference between Deep Learning and PCA (Principal Components Analysis) is that Deep learning uses non-linear function, while PCA uses linear function.

Chapter 15

Applications

```
# Step 1: define a set of function
model = Sequential()
model.add(Dense(input_dim = 28*28,
output_dim = 500))

model.add(Activation('sigmoid'))
model.add(Dense(output_dim = 500))
model.add(Activation('sigmoid'))
model.add(Dense(output_dim=10))
model.add(Activation('softmax'))

# Step 2: goodness of function
model.compile(loss = 'mse',optimizer = SGD(lr=0.1),metrics = ['accuracy'])

# Step 3: Pick the best function
# Step 3.1: Configuration
model.compile(loss = 'mse',optimizer=SGD(lr=0.1),metrics = ['accuracy'])

# Step 3.2: Find the optimal network parameters
model.fit(x_train,y_train,batch_size = 100,nb_epoch=20) # x_train = training data (images)
# y_train: labels (digits)


# Save and load models
# http://keras.io/getting-started/faq/#how-can-i-save-a-keras-model

# How to use the neural network (testing):
# case 1
score = model.evaluate(x_test,y_test)
```

```
print('Total loss on Testing Set:',score[0])
print('Accuracy of Testing set:',score[1])

# case 2
result = model.predict(x_test)
```

Chapter 16

Research Methods

16.1 But-for World

(Mahajan et al., 1993) (Landsman and Stremersch, 2020)

Bibliography

Glorot, X., Bordes, A., and Bengio, Y. (2011). Deep sparse rectifier neural networks. *International Conference on Artificial Intelligence and Statistics*.

Landsman, V. and Stremersch, S. (2020). The commercial consequences of collective layoffs: Close the plant, lose the brand? *Journal of Marketing*, 84(3):122–141.

LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.

Mahajan, V., Sharma, S., and Buzzell, R. D. (1993). Assessing the impact of competitive entry on market expansion and incumbent sales. *Journal of Marketing*, 57(3):39.